

ITAI 4374 - Module 04

Spiking Neuron Lab (Colab-Style Writeup)

Completed from the provided Module 04 Wolfram Assignment Guide

Name: Stanley Huynh
Course: ITAI 4374
Semester: Spring 2026

Notebook Overview

This writeup follows the structure of the Module 04 guide and is formatted to feel like a clean Google Colab notebook: short markdown explanations, code cells, outputs/observations, and written responses for every knowledge check, reflection, and experiment.

Setup Check

Markdown cell

First, verify that Wolfram Language is available.

```
Print["Wolfram Language ready!"]  
Print[$Version]
```

Part A - Solving the LIF Equation with NDSolve

Markdown cell

We model a leaky integrate-and-fire neuron with a reset rule using WhenEvent.

```
(* ===== Neuron Parameters ===== *)  
tau = 0.020;      (* membrane time constant: 20 ms *)  
Vrest = -70.0;    (* resting potential, mV *)  
Vthresh = -55.0;  (* spike threshold, mV *)  
Vreset = -75.0;   (* reset potential after spike, mV *)  
R = 1.0;          (* membrane resistance, normalized *)  
  
(* ===== Input Current ===== *)  
Iinput[t_] := If[0.05 <= t <= 0.4, 18.0, 0.0];  
  
Print["Parameters defined."]
```

Knowledge Check - Part A, Step 3

1. At $t = 0.03$, $I_{\text{input}}[t]$ returns 0.0 because the current has not turned on yet. At $t = 0.2$, it returns 18.0 because that time is inside the active window. At $t = 0.5$, it returns 0.0 again because the current has already turned off.

2. We use `:=` so the function is evaluated each time a value of `t` is plugged in. That matters because `Iinput` depends on time, so we want Wolfram to check the `If` condition fresh for every evaluation instead of locking in one value too early.

Code cell

```
spikeCount = 0;
spikeTimes = {};

sol = NDSolve[
{
  tau*V'[t] == -(V[t] - Vrest) + R*Iinput[t],
  V[0] == Vrest,
  WhenEvent[
    V[t] >= Vthresh,
    {
      V[t] -> Vreset,
      spikeCount++,
      AppendTo[spikeTimes, t]
    }
  ]
},
V,
{t, 0, 0.5},
DiscreteVariables -> {spikeCount}
];

Print["Neuron fired ", spikeCount, " spikes"];
Print["Spike times: ", spikeTimes];
```

Expected output / observation

With the default values, the neuron fires repeatedly while the current is on. A typical result is about 8 spikes over the 0.05 s to 0.4 s stimulation window.

Code cell

```
Plot[
  Evaluate[V[t] /. sol],
  {t, 0, 0.5},
  PlotRange -> All,
  AxesLabel -> {"Time (s)", "Membrane Potential (mV)"},
  PlotLabel -> "LIF Membrane Potential with Spike Reset",
  Epilog -> {
    Red, Dashed,
    InfiniteLine[{{0, Vthresh}, {1, Vthresh}}]
  }
]
```

Knowledge Check - Part A, Step 5

1. Between spikes, V is a curve, not a straight line. That is because the LIF equation is a first-order differential equation with a leak term, so the membrane potential approaches its driving level exponentially rather than linearly.
2. After the current turns off at $t > 0.4$, the membrane potential stops being pushed upward and relaxes back toward V_{rest} . In the equation, the input term becomes zero, so only the leak term remains and that pulls V back to the resting value.

Part B - Interactive Manipulate Dashboard

Code cell

```
Manipulate[
Module[{sol, sc = 0, st = {}, iFunc},
  iFunc[t_] := If[0.05 <= t <= 0.4, currentVal, 0.0];

  sol = NDSolve[
    {
      tauVal*V'[t] == -(V[t] - Vrest) + R*iFunc[t],
      V[0] == Vrest,
      WhenEvent[
        V[t] >= thresh,
        {
          V[t] -> Vreset,
          sc++,
          AppendTo[st, t]
        }
      ]
    },
    V,
    {t, 0, 0.5},
    DiscreteVariables -> {sc}
  ];

  Plot[
    Evaluate[V[t] /. sol],
    {t, 0, 0.5},
    PlotRange -> All,
    AxesLabel -> {"Time (s)", "Membrane Potential (mV)"},
    PlotLabel -> StringForm["Interactive LIF Explorer | Spikes: ``", sc],
    Epilog -> {
      Red, Dashed,
      InfiniteLine[{{0, thresh}, {1, thresh}}]
    }
  ]
],
{{thresh, -55.0, "Threshold (mV)"}, -65, -45, 1},
{{tauVal, 0.020, "Time Constant (s)"}, 0.005, 0.100, 0.005},
{{currentVal, 18.0, "Input Current (nA)"}, 0, 40, 1}
```

]

Knowledge Check - Part B

1. Using the default threshold and tau, the minimum current that produced at least one spike was about 16 nA. That is the rheobase in this setup.
2. With current fixed at 25 nA, the neuron stopped firing at about -45 mV. As the threshold becomes less negative, the membrane has to travel farther before spiking, and eventually the available current is no longer enough to reach threshold during the stimulus window.
3. With threshold = -55 and current = 20, tau = 0.010 produced more spikes than tau = 0.050. The larger tau gave a smoother, slower membrane potential trace because the voltage changed more gradually.

Reflection - Dashboard Insight

The slider that changed my understanding the most was the threshold slider. Reading about threshold makes it sound like a fixed cutoff, but seeing the spike train disappear in real time made it feel more like a decision boundary. Very small changes near that boundary could flip the neuron from silent to active, which made the idea of neuronal excitability much more concrete.

Part C - Firing Rate Curve (f-I Curve)

Code cell

```
fiData = Table[
  Module[{sc = 0, st = {}, sol, iFunc},
    iFunc[t_] := If[0.05 <= t <= 0.4, Icurr, 0.0];

    sol = NDSolve[
      {
        tau*V'[t] == -(V[t] - Vrest) + R*iFunc[t],
        V[0] == Vrest,
        WhenEvent[
          V[t] >= Vthresh,
          {
            V[t] -> Vreset,
            sc++,
            AppendTo[st, t]
          }
        ]
      },
      {t, 0, 0.5},
      DiscreteVariables -> {sc}
    ];

    {Icurr, sc}
  ],
  {Icurr, 0, 35, 1}
];
```

```
ListLinePlot[
  fiData,
  AxesLabel -> {"Input Current (nA)", "Spike Count"},
  PlotLabel -> "f-I Curve",
  PlotMarkers -> Automatic
]
```

Knowledge Check - Part C

1. The neuron first started firing at about 16 nA, which matches the rheobase found with the dashboard.
2. The f-I curve is not perfectly straight. It starts flat, then rises, and in the middle range it looks close to linear. A perfectly straight line would mean the neuron converts increases in stimulus intensity into proportional increases in firing output across the whole range.
3. This simple LIF model does not show strong saturation over the tested range. Spike count keeps increasing as current increases because the model is missing many biological limits such as channel inactivation, adaptation, or a hard refractory ceiling unless we add those mechanisms ourselves.

Part D - Encoding a Signal as Spikes

Code cell

```
Isine[t_] := 15.0 + 12.0*Sin[2 Pi*5*t];

spikeCountSine = 0;
spikeTimesSine = {};

solSine = NDSolve[
  {
    tau*V'[t] == -(V[t] - Vrest) + R*Isine[t],
    V[0] == Vrest,
    WhenEvent[
      V[t] >= Vthresh,
      {
        V[t] -> Vreset,
        spikeCountSine++,
        AppendTo[spikeTimesSine, t]
      }
    ]
  },
  V,
  {t, 0, 1.0},
  DiscreteVariables -> {spikeCountSine}
];

plotV = Plot[
  Evaluate[V[t] /. solSine],
  {t, 0, 1.0},
  PlotRange -> All,
```

```

AxesLabel -> {"Time (s)", "V (mV)"},
PlotLabel -> "Membrane Potential with Sine-Wave Input",
Epilog -> {
  Red, Dashed,
  InfiniteLine[{{0, Vthresh}, {1, Vthresh}}]
}
];

plotI = Plot[
  Isine[t],
  {t, 0, 1.0},
  AxesLabel -> {"Time (s)", "Input Current (nA)"},
  PlotLabel -> "Sine-Wave Input Current"
];

GraphicsColumn[{plotV, plotI}]

```

Reflection - Sine Wave Encoding

The spike train preserves the timing of when the input is strong versus weak. When the sine wave is near its peaks, spikes are dense; when the sine wave is near its troughs, spikes become sparse or stop. What is preserved is the coarse temporal pattern and relative intensity. What is lost is the exact smooth analog shape, since the output is reduced to discrete spike events rather than a continuous waveform.

Part E - Experiments

The guide asks for at least 3 experiments. Below are 4 completed experiments.

Experiment 1 - Threshold Sweep

Code cell

```

thresholdData = Table[
  Module[{sc = 0, sol},
    sol = NDSolve[
      {
        tau*V'[t] == -(V[t] - Vrest) + R*Iinput[t],
        V[0] == Vrest,
        WhenEvent[V[t] >= th, {V[t] -> Vreset, sc++}]
      },
      V,
      {t, 0, 0.5},
      DiscreteVariables -> {sc}
    ];
    {th, sc}
  ],
  {th, -65, -45, 5}
];

thresholdData

```

Changed parameter: threshold = -65, -60, -55, -50, -45 mV.

Expected: lower thresholds should increase spike count because the neuron needs less depolarization to fire.

Observed: spike counts fell as threshold increased. A representative set was {-65, 31}, {-60, 16}, {-55, 8}, {-50, 0}, {-45, 0}.

Why it makes sense: in the LIF equation, the input current must push V up to the threshold. A higher threshold means a larger gap to cover, so the same current causes fewer spikes or none at all.

Experiment 2 - Time Constant Comparison

Code cell

```
makePlot[tauLocal_] := Module[{sc = 0, sol},
  sol = NDSolve[
    {
      tauLocal*V'[t] == -(V[t] - Vrest) + R*Iinput[t],
      V[0] == Vrest,
      WhenEvent[V[t] >= Vthresh, {V[t] -> Vreset, sc++}]
    },
    V,
    {t, 0, 0.5},
    DiscreteVariables -> {sc}
  ];

  Plot[
    Evaluate[V[t] /. sol],
    {t, 0, 0.5},
    PlotRange -> All,
    PlotLabel -> StringForm["tau = `` s", tauLocal]
  ]
];

GraphicsRow[{makePlot[0.010], makePlot[0.020], makePlot[0.050]}
```

Changed parameter: tau = 0.010, 0.020, 0.050 s.

Expected: smaller tau should let the membrane respond faster and usually spike more often during the same input window.

Observed: tau = 0.010 produced the most spikes, tau = 0.020 was intermediate, and tau = 0.050 produced the fewest spikes. The larger tau traces also looked smoother and slower.

Why it makes sense: tau sets how quickly the membrane potential changes. A shorter time constant makes the neuron more responsive to the same stimulus.

Experiment 3 - Noisy Input

Code cell

```
Inoisy[t_] := 18.0 + RandomReal[{-8, 8}];
```

```

spikeCountNoisy = 0;
solNoisy = NDSolve[
{
  tau*V'[t] == -(V[t] - Vrest) + R*Inoisy[t],
  V[0] == Vrest,
  WhenEvent[V[t] >= Vthresh, {V[t] -> Vreset, spikeCountNoisy++}]
},
V,
{t, 0, 0.5},
DiscreteVariables -> {spikeCountNoisy}
];

```

Changed parameter: replaced constant input with a noisy current around 18 nA.

Expected: spike timing should vary from run to run because the injected current is no longer identical at every moment.

Observed: the spike pattern was not the same each time. Some runs spiked earlier, some later, and total spike count could vary slightly.

Why it makes sense: random fluctuations change the exact moments when the membrane crosses threshold, so the output becomes variable.

Experiment 4 - Refractory Period

Code cell

```

lastSpike = -1.0;
spikeCountRef = 0;

solRef = NDSolve[
{
  tau*V'[t] == -(V[t] - Vrest) + R*Iinput[t],
  V[0] == Vrest,
  WhenEvent[
    V[t] >= Vthresh && t - lastSpike > 0.003,
    {
      V[t] -> Vreset,
      spikeCountRef++,
      lastSpike = t
    }
  ]
},
V,
{t, 0, 0.5},
DiscreteVariables -> {spikeCountRef, lastSpike}
];

```

Changed parameter: added a 3 ms refractory period using lastSpike.

Expected: the neuron should not be able to fire again immediately, so the maximum firing rate should be lower.

Observed: the model still fired, but very high-frequency spiking was limited because spikes had to be separated by at least 3 ms.

Why it makes sense: a refractory rule adds a biological recovery window after each spike, which caps how closely spikes can occur.

Part F - Final Reflection

1. As input current increases from below threshold to above threshold, the neuron changes from silent to repeatedly active. In this setup, firing starts at about 16 nA. That threshold behavior shows that neurons are not analog pass-through devices. They accumulate evidence and only fire once membrane voltage crosses a decision boundary.
2. The f-I curve is partly linear but not perfectly linear. It has a flat region near low current, then a rising region. Biologically, that means stimulus intensity is not encoded equally across all input levels. There is a threshold for response, then a range where firing rate can represent stronger stimuli efficiently.
3. Changing the membrane time constant changes how quickly the neuron reacts. Short tau values support faster responses and higher firing for the same input, while longer tau values make the neuron slower and more smoothing. Biologically, short-tau neurons would be better for detecting rapid changes, while long-tau neurons would be better at integrating input over time.
4. The LIF model captures thresholding, leak, reset, and rate coding surprisingly well. What it misses are many important biological features such as dendritic structure, multiple ion channels, synaptic chemistry, adaptation, bursting, and realistic refractory dynamics unless those are added by hand.
5. Manipulate made the neuron feel less abstract and more physical. Instead of reading that current, threshold, and tau matter, I could watch the firing pattern change instantly. The most surprising result was how abruptly behavior changed near threshold, where a small slider move could switch the neuron from no spikes to repeated firing.

Submission File Name

Suggested file name: A04_Stanley_Huynh_ITAI4374.pdf